

Logical Design

Zvi Kohavi and Niraj K. Jha

1

Design with Basic Logic Gates

Logic gates: perform logical operations on input signals

Positive (negative) logic polarity: constant 1 (0) denotes a high voltage and constant 0 a low (high) voltage

Synchronous circuits: driven by a clock that produces a train of equally spaced pulses

Asynchronous circuits: are almost free-running and do not depend on a clock; controlled by initiation and completion signals

Fanout: number of gate inputs driven by the output of a single gate

Fanin: bound on the number of inputs a gate can have

Propagation delay: time to propagate a signal through a gate

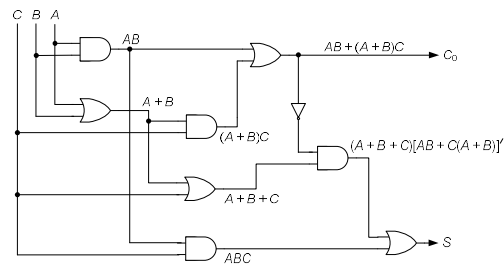
2

Analysis of Combinational Circuits

Circuit analysis: determine the Boolean function that describes the circuit

- Done by tracing the output of each gate, starting from circuit inputs and continuing towards each circuit output

Example: a multi-level realization of a full binary adder



$$C_0 = AB + (A + B)C$$

$$= AB + AC + BC$$

$$S = (A + B + C)[AB + (A + B)C]' + ABC$$

$$= (A + B + C)(A' + B')(A' + C')(B' + C') + ABC$$

$$= AB'C' + A'BC' + A'B'C + ABC$$

$$= A \oplus B \oplus C$$

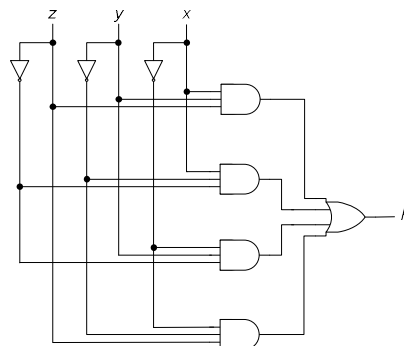
3

Simple Design Problems

Parallel parity-bit generator: produces output value 1 if and only if an odd number of its inputs have value 1

z \ xy	00	01	11	10
0	0	1	0	1
1	1	0	1	0

(a) Map.



(b) Implementation.

$$P = x'y'z + x'yz' + xy'z' + xyz$$

4

Logic Design with Integrated Circuits

Small scale integration (SSI): integrated circuit packages containing a few gates; e.g., AND, OR, NOT, NAND, NOR, XOR

Medium scale integration (MSI): packages containing up to about 100 gates; e.g., code converters, adders

Large scale integration (LSI): packages containing thousands of gates; arithmetic unit

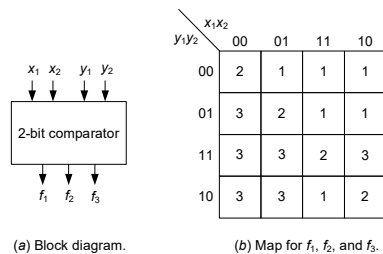
Very large scale integration (VLSI): packages with millions of gates

5

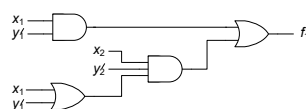
Comparators

n -bit comparator: compares the **magnitude** of two numbers X and Y , and has three outputs f_1 , f_2 , and f_3

- $f_1 = 1$ iff $X > Y$
- $f_2 = 1$ iff $X = Y$
- $f_3 = 1$ iff $X < Y$



$$\begin{aligned}
 f_1 &= x_1x_2y_2' + x_2y_1'y_2' + x_1y_1' \\
 &= (x_1 + y_1)x_2y_2' + x_1y_1' \\
 f_2 &= x_1'x_2'y_1'y_2' + x_1'x_2y_1'y_2 + \\
 &\quad x_1x_2'y_1y_2' + x_1x_2y_1y_2 \\
 &= x_1'y_1'(x_2'y_2' + x_2y_2) + \\
 &\quad x_1y_1(x_2'y_2' + x_2y_2) \\
 &= (x_1'y_1' + x_1y_1)(x_2'y_2' + x_2y_2) \\
 f_3 &= x_2'y_1y_2 + x_1'x_2'y_2 + x_1'y_1 \\
 &= x_2'y_2(y_1 + x_1') + x_1'y_1
 \end{aligned}$$



6

Comparators (Contd.)

How to design an n -bit comparator?

A	B	Y (A > B)	Y(A=B)	Y(A<B)
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

$$Y(A>B) = AB'$$

$$Y(A=B) = AB + A'B'$$

$$Y(A<B) = A'B$$

$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

$$x_i = A_i B_i + A'_i B'_i \quad \text{for } i = 0, 1, 2, 3$$

$$(A = B) = x_3 x_2 x_1 x_0$$

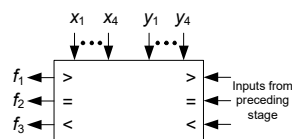
$$(A > B) = A_3 B'_3 + x_3 A_2 B'_2 + x_3 x_2 A_1 B'_1 + x_3 x_2 x_1 A_0 B'_0$$

7

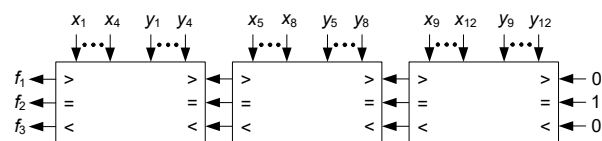
4-bit/12-bit Comparators

Four-bit comparator: 11 inputs (four for X , four for Y , and three connected to outputs f_1 , f_2 and f_3 of the preceding stage)

12-bit comparator:



(a) A 4-bit comparator.



(b) A 12-bit comparator.

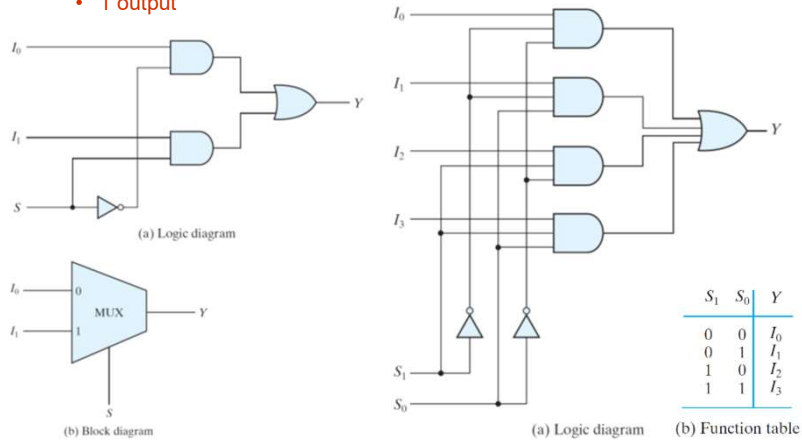
8

Data Selectors

Multiplexer: electronic switch that connects one of n inputs to the output

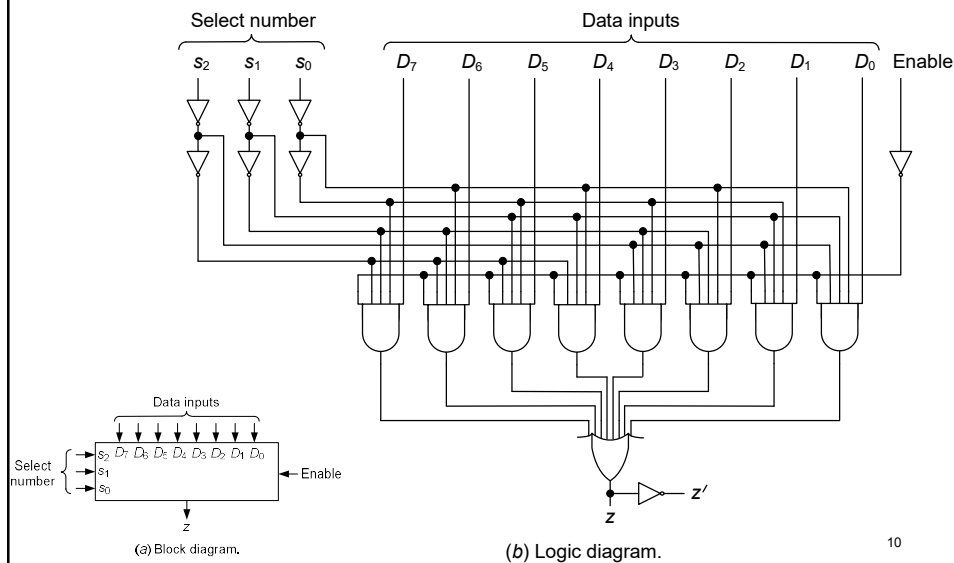
Data selector: application of multiplexer

- n data input lines, $D_0, D_1, \dots, D_{n-1}$
- m select digit inputs $s_0, s_1, \dots, s_{m-1}$
- 1 output



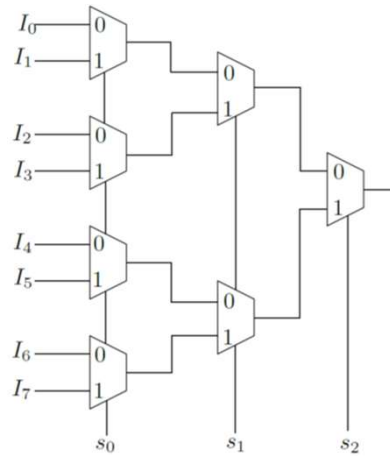
9

Data Selectors



10

Cascading of Data Selectors

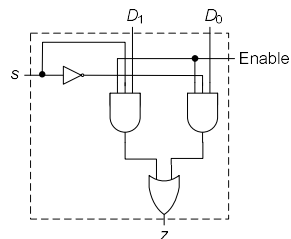


11

Implementing Switching Functions with Data Selectors

Data selectors: can implement arbitrary switching functions

Example: implementing two-variable functions



$$z = sD_1 + s'D_0$$

If $s = A$, $B = D_0$, and $B' = D_1$, then $z = A \oplus B$.

If $s = A$, $D_0 = 1$, and $D_1 = B'$, then $z = A' + B'$.

To implement an n -variable function: a data selector with $n-1$ select inputs and 2^{n-1} data inputs

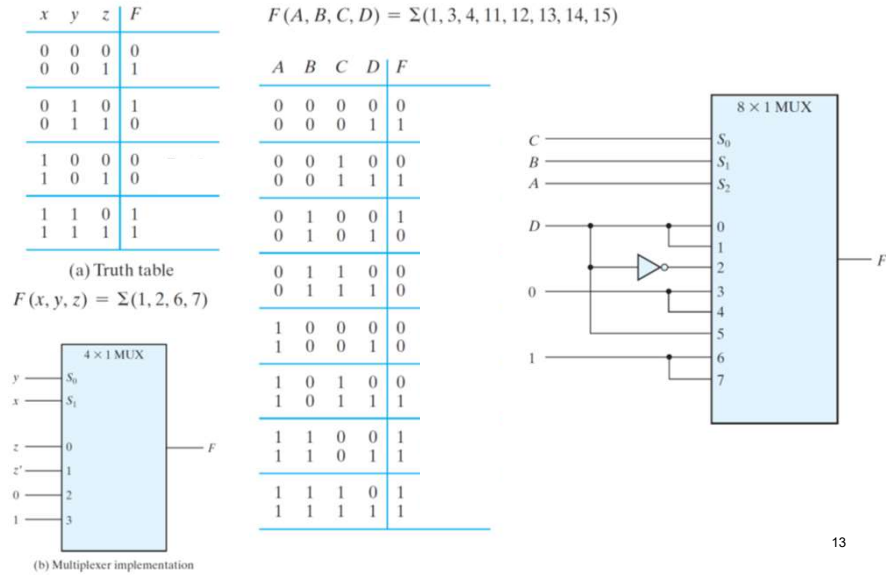
Implementing three-variable functions:

$$z = s_2's_1'D_0 + s_2's_1D_1 + s_2s_1'D_2 + s_2s_1D_3$$

General case: Assign $n-1$ variables to the select inputs and last variable and constants 0 and 1 to the data inputs such that desired function results

12

Implementing Switching Functions with Data Selectors (Contd.)

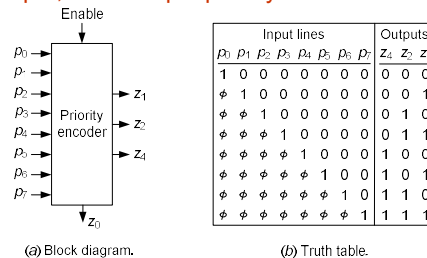


Priority Encoders

Priority encoder: n input lines and $\log_2 n$ output lines

- Input lines represent units that may request service
- When inputs p_i and p_j , such that $i > j$, request service simultaneously, line p_i has priority over line p_j
- Encoder produces a binary output code indicating which of the input lines requesting service has the highest priority

Example: Eight-input, three-output priority encoder

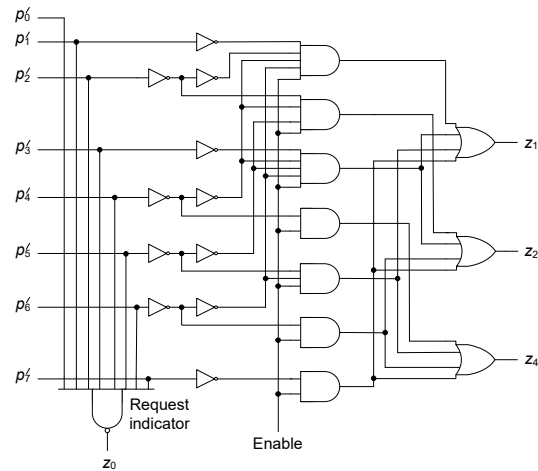


$$z_4 = p_4 p_5 p_6 p_7' + p_5 p_6 p_7' + p_6 p_7' + p_7 = p_4 + p_5 + p_6 + p_7$$

$$z_2 = p_2 p_3 p_4 p_5 p_6 p_7' + p_3 p_4 p_5 p_6 p_7' + p_6 p_7' + p_7 = p_2 p_4 p_5' + p_3 p_4 p_5' + p_6 + p_7$$

$$z_1 = p_1 p_2 p_3 p_4 p_5 p_6 p_7' + p_3 p_4 p_5 p_6 p_7' + p_6 p_6 p_7' + p_7 = p_1 p_2 p_4 p_6' + p_3 p_4 p_6' + p_5 p_6' + p_7$$

Priority Encoders (Contd.)



(c) Logic diagram.

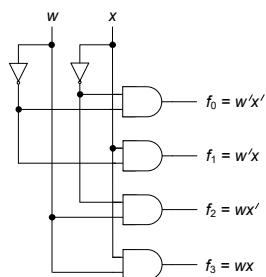
15

Decoders

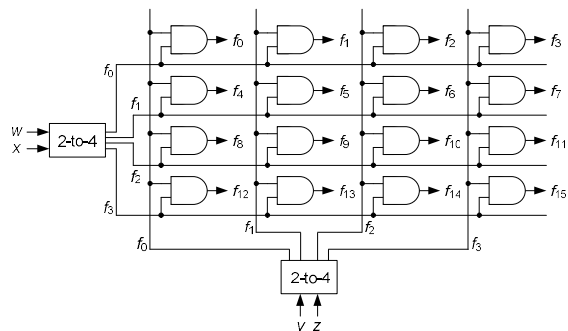
Decoders with n inputs and 2^n outputs: for any input combination, only one output is 1

Useful for:

- Routing input data to a specified output line, e.g., in addressing memory
- Basic building blocks for implementing arbitrary switching functions



2-to-4- decoder

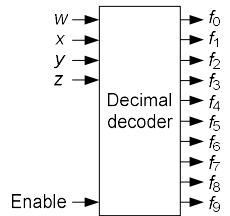


4-to-16 decoder made of two 2-to-4 decoders and a gate-switching matrix

16

Decimal Decoder

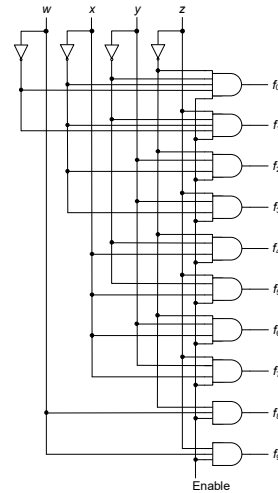
BCD-to-decimal: 4-to-16 decoder made of two 2-to-4 decoders and a gate-switching matrix



(a) Block diagram.

wx \ yz	00	01	11	10
00	f_0	f_4	ϕ	f_8
01	f_1	f_5	ϕ	f_9
11	f_3	f_7	ϕ	ϕ
10	f_2	f_6	ϕ	ϕ

(b) Map.

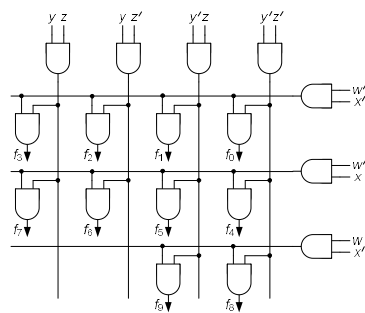


(c) Logic diagram.

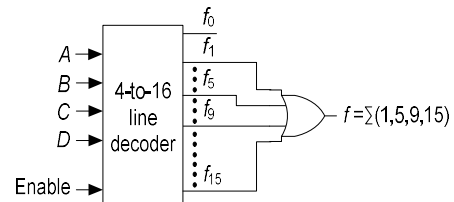
17

Decimal Decoder (Contd.)

Implementation using a partial-gate matrix:



Implementing switching function

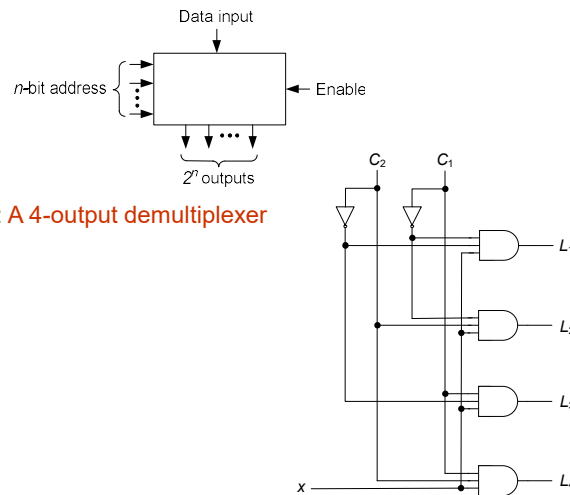


18

Demultiplexers

Demultiplexers: decoder with 1 data input and n address inputs

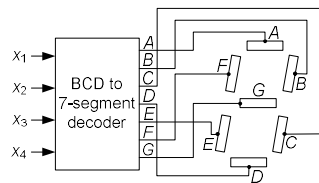
- Directs input to any one of the 2^n outputs



19

Seven-segment Display

Seven-segment display: BCD to seven-segment decoder and seven LEDs



Seven-segment pattern and code:

Decimal digit	BCD code				Seven-segment code						
	x_1	x_2	x_3	x_4	A	B	C	D	E	F	G
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	0	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	0	0	1	1
0	0	0	0	0	1	1	1	1	1	1	0

$$A = x_1 + x_2'x_4' + x_2x_4 + x_3x_4$$

$$B = x_2' + x_3'x_4' + x_3x_4$$

$$C = x_2 + x_3' + x_4$$

$$D = x_2'x_4' + x_2'x_3 + x_3x_4' + x_2x_3'x_4$$

$$E = x_2'x_4' + x_3x_4'$$

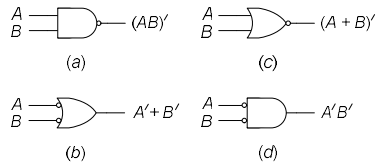
$$F = x_1 + x_2x_3' + x_2x_4' + x_3'x_4'$$

$$G = x_1 + x_2'x_3 + x_2x_3' + x_3x_4'$$

20

NAND/NOR Circuits

Switching algebra: not directly applicable to NAND/NOR logic

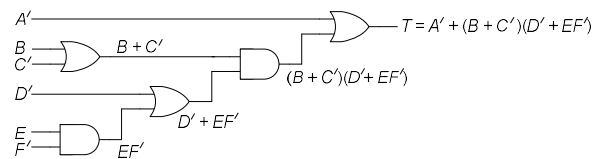
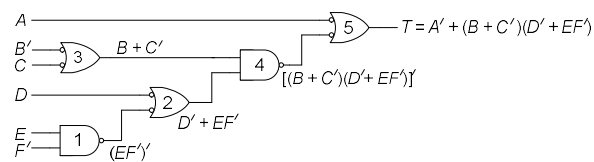


NAND and NOR gate symbols

21

Analysis of NAND/NOR Networks

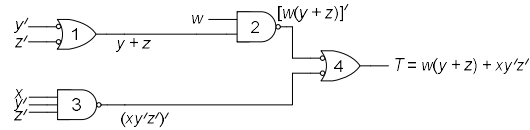
Example: circles (inversions) at both ends of a line cancel each other



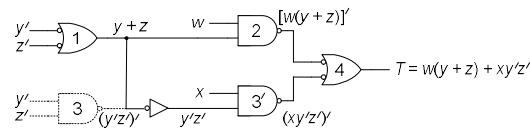
22

Synthesis of NAND/NOR Networks

Example: Realize $T = w(y+z) + xy'z'$



(a) First realization.



(b) Realization with two-input gates.

23

Design of High-speed Adders

Full adder: performs binary addition of three binary digits

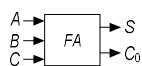
- Inputs: arguments A and B and carry-in C
- Outputs: sum S and carry-out C_0

Example:

$0\ 1\ 1$ = carry-in
 $1\ 0\ 1\ 1$ = augend
 $0\ 0\ 1\ 1$ = addend
 $\hline 1\ 1\ 1\ 0$ = sum

Truth table, block diagram and expressions:

A	B	C	S	C ₀
0	0	0	0	0
0	0	1	1	0
0	1	1	0	1
0	1	0	1	0
1	1	0	0	1
1	1	1	1	1
1	0	1	0	1
1	0	0	1	0



(b) Block diagram.

$$\begin{aligned}
 S &= A'B'C + A'BC' + AB'C' + ABC \\
 &= A \oplus B \oplus C \\
 C_0 &= A'BC + ABC' + AB'C + ABC \\
 &= AB + AC + BC
 \end{aligned}$$

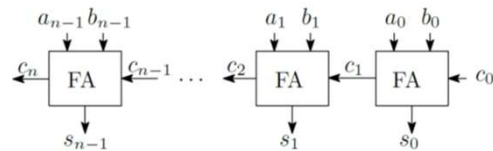
(a) Truth table for S and C_0 .

24

Ripple-carry Adder

Ripple-carry adder: Stages of full adders

- C_i : input carry
- C_{i+1} : output carry



$$s_i = a_i \oplus b_i \oplus c_i$$

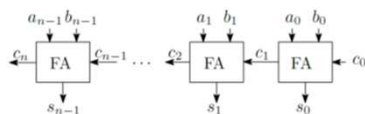
$$c_{i+1} = a_i b_i + a_i c_i + b_i c_i$$

Time required:

- Time per full adder: 2 units
- Time for ripple-carry adder: $2n$ units

25

Carry-lookahead Adder



$$c_1 = b_0 c_0 + a_0 c_0 + a_0 b_0$$

$$c_2 = b_1 c_1 + a_1 c_1 + a_1 b_1$$

$$c_2 = a_1 a_0 b_0 + a_1 a_0 c_0 + a_1 b_0 c_0 + b_1 a_0 b_0 + b_1 a_0 c_0 + b_1 b_0 c_0 + a_1 b_1$$

Faster generation of carry

$$c_{i+1} = b_i c_i + a_i c_i + a_i b_i$$

$$= a_i b_i + (a_i + b_i) c_i$$

$$c_2 = a_1 b_1 + (a_1 + b_1)(a_0 b_0 + (a_0 + b_0) c_0)$$

$$g_i = a_i b_i$$

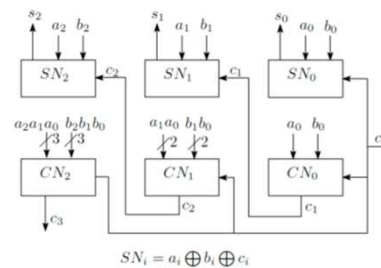
$$p_i = a_i + b_i$$

$$c_{i+1} = g_i + p_i c_i$$

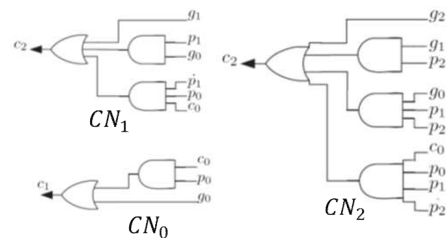
$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$$

$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$



$$SN_i = a_i \oplus b_i \oplus c_i$$



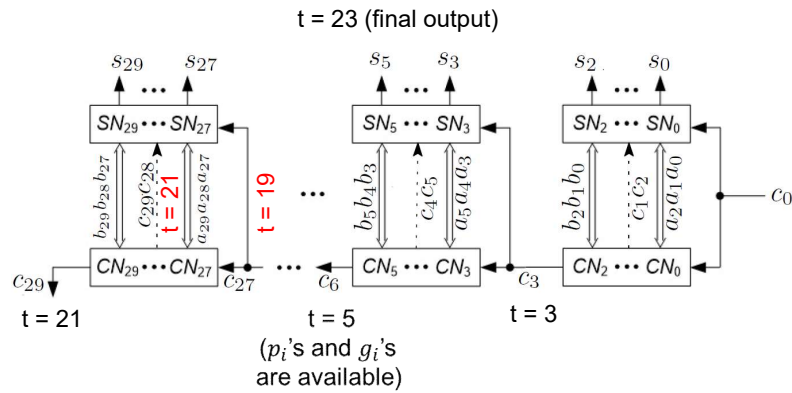
delay = 3

26

30-bit Adder

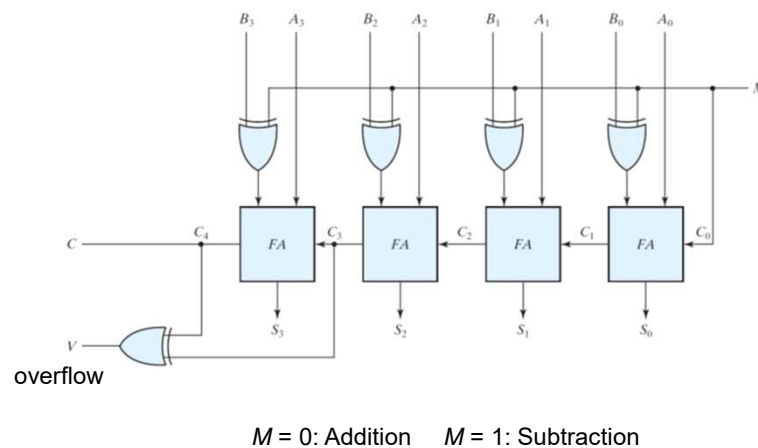
Example: divide n stages into groups of three stages

- Time taken: $3 + 2n/3$ time units
- 50% additional hardware for a threefold speedup



27

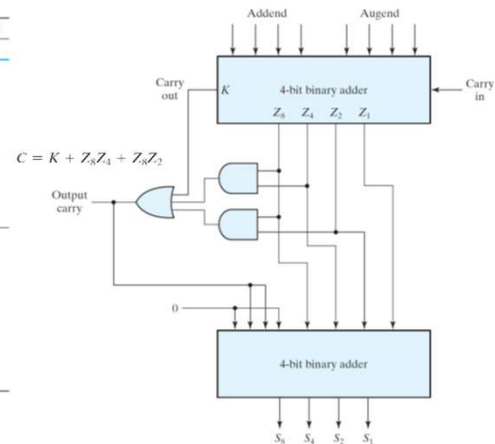
4-bit Adder-Subtractor



28

BCD Adder

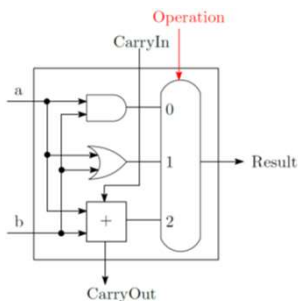
Binary Sum					BCD Sum					Decimal
K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19



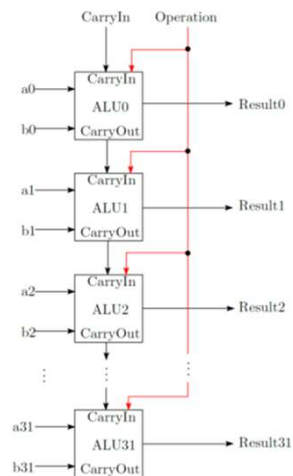
29

ALU Design

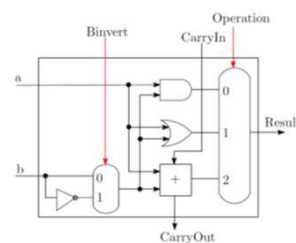
1bit ALU:



Q. How to design 32-bit ALU from 1-bit ALU?

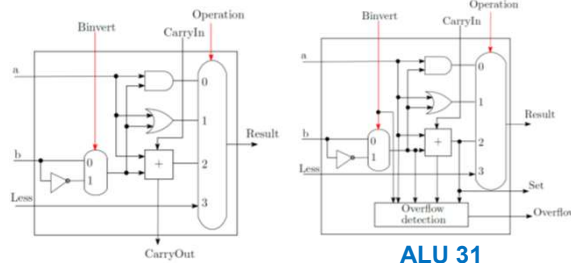


Q. How to do subtraction?



Tailoring the 32-bit ALU

Compare signed numbers: Idea: $(a-b) < 0 \rightarrow a < b$

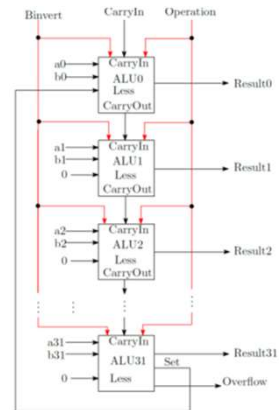


ALU 0 – ALU 30

ALU 31

Problem: Overflow e.g., $rs = (1001)_2 = -7$ and $rt = (0110)_2 = 6$

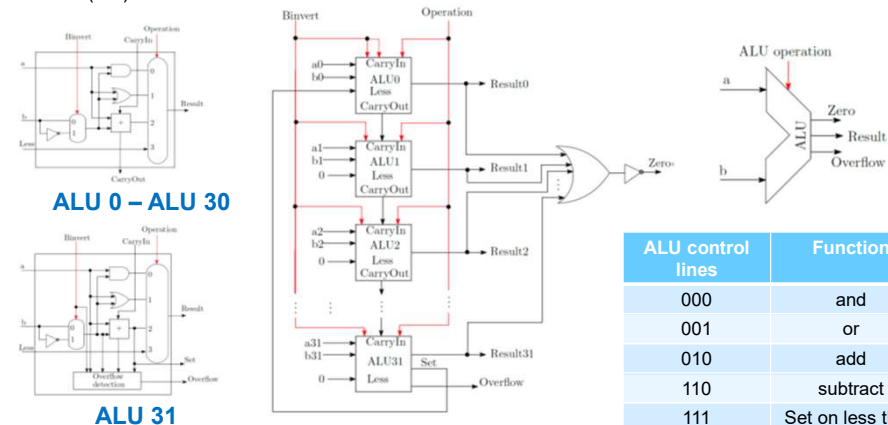
How to modify the 1-bit ALU to handle set-less-than correctly?



Tailoring the 32-bit ALU

Supporting conditional branch: beq rs, rt, L1 -- if $(rs == rt)$ branch to instruction labeled L1

Idea: $(a-b) = 0 \rightarrow a = b$



ALU 0 – ALU 30

ALU 31

ALU control lines	Function
000	and
001	or
010	add
110	subtract
111	Set on less than

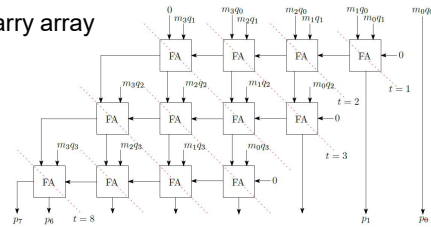
Unsigned Multiplication (Combinational)

1 1 0 1
 Multiplicand (13)
 1 0 1 1
 Multiplier (11)

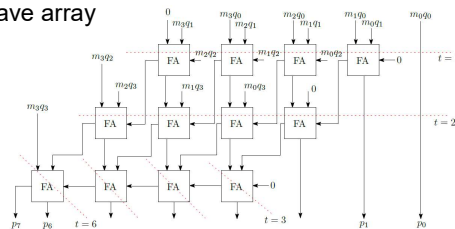
 1 1 0 1
 1 1 0 1
 0 0 0 0
 1 1 0 1

	m_3	m_2	m_1	m_0			
	q_3	q_2	q_1	q_0			
	m_3q_0	m_2q_0	m_1q_0	m_0q_0			
	m_3q_1	m_2q_1	m_1q_1	m_0q_1			
	m_3q_2	m_2q_2	m_1q_2	m_0q_2			
	m_3q_3	m_2q_3	m_1q_3	m_0q_3			
p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0

Ripple-carry array



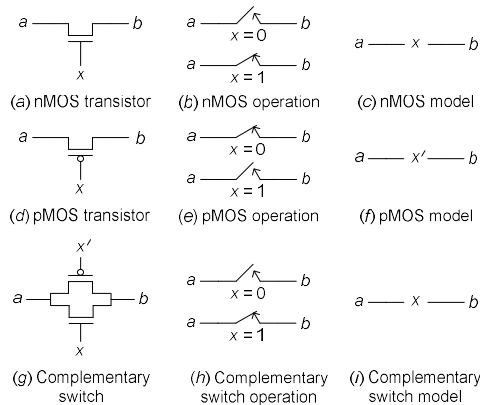
Carry-save array



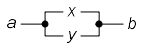
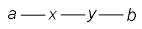
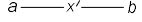
Metal-oxide Semiconductor (MOS) Transistors and Gates

Complementary metal-oxide semiconductor (CMOS): currently the dominant technology

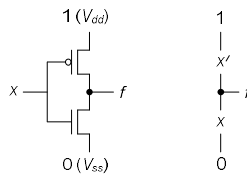
- Two types of transistors: nMOS and pMOS



Transmission Function of a Network

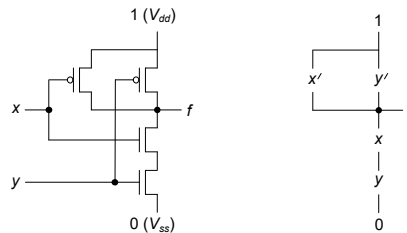
Network	Transmission function
	$T_{ab} = x + y$
	$T_{ab} = xy$
	$T_{ab} = x'$

CMOS inverter and its transmission functions:

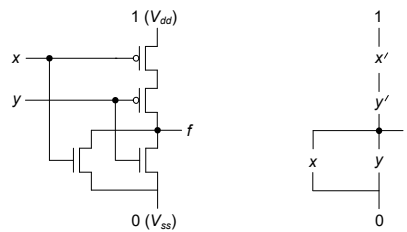


35

CMOS NAND/NOR Gates



(a) CMOS NAND gate and its transmission functions.



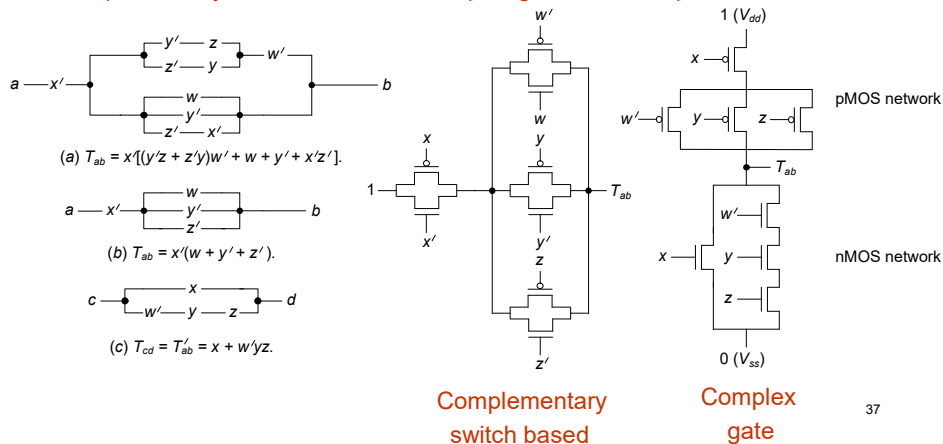
(b) CMOS NOR gate and its transmission functions.

36

Analysis of Series-parallel Networks

Algebra of MOS networks: isomorphic to switching algebra

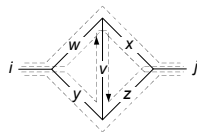
Example: Find the transmission function of the network and its complementary switch based and complex gate CMOS implementations



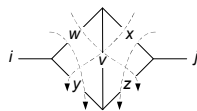
Analysis of Non-series-parallel Networks

Obtaining the transmission function:

- Tie sets: minimal paths between two terminals
- Cut sets: minimal sets of branches, when open, ensure no transmission between the two terminals



(a) Tie sets. $T_{ij} = wx + wz + yvx + yz$.



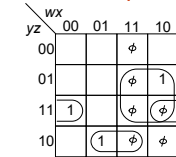
(b) Cut sets. $T_{ij} = (w + y)(w + v + z)(x + v + y)(x + z)$.

Synthesis of MOS Networks

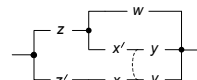
Sneak paths in non-series-parallel networks: undesired paths that may change the transmission function

- Occur because of bilateral nature of MOS transistors

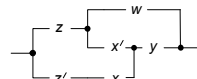
Example: Design a minimal network with BCD inputs that produces a 1 whenever the input is 3 or a multiple of 3



(a) Map for $T = wz + xyz' + x'yz$.



(b) Series-parallel realization of T .



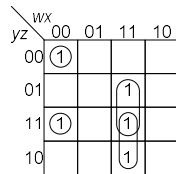
(c) Minimal realization of T .

Sneak path: $z'xx'w$ – OK since it has no effect on the transmission function

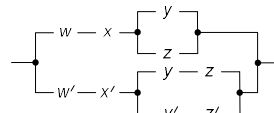
39

Synthesis of MOS Networks (Contd.)

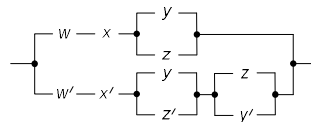
Example: Design a minimal network to realize $T(w,x,y,z) = \sum(0,3,13,14,15)$



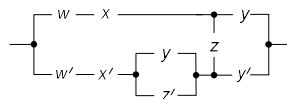
(a) Map for $T = wx y + wx z + w'x'y'z' + w'x'yz$.



(b) Series-parallel realization of T .



(c) An alternative series-parallel realization of T .



(d) A minimum realization of T .

40